MODULE 11 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Administering a cluster

Building clusters, upgrading them, maintaining nodes, and sharing capacity fairly.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 11 of 35

admin@sthithapragna.com

Good administration is **changing things without anyone noticing**: upgrades, maintenance, and fair shares.

Ways to get a cluster

OPTION	WHO RUNS THE CONTROL PLANE	TYPICAL USE
Managed: EKS, GKE, AKS	The cloud provider	Most production clusters
kubeadm	You	Self-managed clusters, learning the internals
k3s, k0s, MicroK8s	You, in a lightweight package	Edge, IoT, small machines
kind, minikube	Your laptop	Local development and CI tests
OpenShift, Rancher	A platform on top of Kubernetes	Enterprise platforms

ON THE EXAM

The CNCF's Certified Kubernetes programme checks that a distribution passes the conformance tests.

kubeadm: building a cluster yourself

- **kubeadm init** sets up a control plane: certificates, kubeconfig files, static Pod manifests.
- **kubeadm join** adds nodes, using a bootstrap token printed by init.
- **kubeadm installs no network plugin.** Until you add a CNI plugin, nodes stay NotReady.
- **kubeadm upgrade plan / apply** upgrades the control plane, one minor version at a time.
- Its certificates last one year; **kubeadm certs** checks and renews them, and upgrades renew them too.

> ON THE EXAM
>
> Fresh kubeadm cluster, nodes NotReady, CoreDNS Pending: the CNI plugin is missing.

A kubeadm cluster in three commands

```
1 # on the first control plane node
2 $ kubeadm init --pod-network-cidr=10.244.0.0/16
3
4 $ kubectl apply -f <your CNI plugin's manifest>
5
6 # on each worker node
7 $ kubeadm join 10.0.0.10:6443 --token abcdef.0123456789abcdef \
8     --discovery-token-ca-cert-hash sha256:<hash from init>
```

2 Builds the control plane and prints the join command.

4 The step people forget: install a network plugin.

7-8 Token proves the node may join; the hash proves the cluster is genuine.

How Kubernetes releases

- **About three minor releases a year,** one every four months or so.
- **The three most recent minor versions are maintained:** today 1.37, 1.36 and 1.35.
- **Patch releases** such as 1.37.2 carry fixes, including security fixes.
- **Fall behind and you lose patches.** Plan an upgrade at least once a year.

> ON THE EXAM
>
> Versions read major.minor.patch. Upgrades are planned by minor version.

Version skew: what may run together

COMPONENT	ALLOWED VERSIONS, RELATIVE TO THE API SERVER
`kube-apiserver (HA)`	All instances within one minor version of each other
`controller-manager, scheduler`	Not newer; at most one minor older
`kubelet`	Not newer; up to three minors older
`kube-proxy`	Not newer; up to three minors older
`kubectl`	One minor older or newer

ON THE EXAM

With the API server at 1.37, kubelets may run 1.34 to 1.37. Never newer.

The upgrade order

01

Back up etcd

A snapshot, stored off the cluster.

02

API servers

First, one minor version at a time.

03

Other control plane

Controller manager and scheduler.

04

Nodes, one by one

Drain, upgrade kubelet, uncordon.

05

Add-ons

CNI, CoreDNS, and so on, per their docs.

ON THE EXAM

Control plane before nodes, never skip a minor version, and back up etcd first.

cordon versus drain

KUBECTL CORDON

Close to new bookings

- Node marked SchedulingDisabled
- No new Pods are placed there
- Existing Pods keep running
- Reverse with kubectl uncordon

KUBECTL DRAIN

Move the diners out

- Cordons the node first
- Then evicts its Pods gracefully
- Respects PodDisruptionBudgets
- Node is then safe to work on

THE TIEBREAKER

Cordon stops new arrivals. Drain also moves the current residents. Both are undone with uncordon.

Maintaining a node

```
$ kubectl drain node-3 --ignore-daemonsets --delete-emptydir-data
node/node-3 cordoned
evicting pod quayside/web-7d9f6c-x2k4p
evicting pod quayside/booking-api-5b8c4d-q8r2m
node/node-3 drained

# patch, reboot, upgrade the kubelet...

$ kubectl uncordon node-3
node/node-3 uncordoned
```

1. DaemonSet Pods cannot move, so drain must be told to ignore them.
2. Drain cordons first, automatically.
9. Nothing moves back on its own. New Pods may now land here.

PodDisruptionBudgets

- **A promise about availability:** keep at least N Pods, or lose at most N, for a label selector.
- **Drain uses the Eviction API,** which refuses an eviction that would break the budget.
- **Only voluntary disruptions** are covered: drains, upgrades, autoscaler scale-down.
- **A budget that allows zero disruptions** blocks every drain until someone intervenes.

> ON THE EXAM
>
> minAvailable equal to the replica count means no Pod may ever be evicted. Leave headroom.

A disruption budget for the booking API

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: booking-api-pdb}
spec:
  minAvailable: 2
  selector: {matchLabels: {app: booking-api}}
```

5 Or maxUnavailable. A number or a percentage.

6 Which Pods the budget protects.

THE TAKEAWAY

With 3 replicas this allows one eviction at a time. With 2 replicas, it allows none.

Voluntary versus involuntary disruptions

VOLUNTARY

Someone chose it

- kubectl drain
- Cluster or node upgrades
- Autoscaler removing a node
- PDBs apply

INVOLUNTARY

It just happened

- Hardware or kernel failure
- A cloud VM disappearing
- Node running out of resources
- PDBs cannot prevent it

THE TIEBREAKER

Budgets manage planned change. Protection from failure comes from replicas spread across nodes and zones.

Teams share one cluster. Quotas and limit ranges keep one team from starving the rest.

ResourceQuota versus LimitRange

RESOURCEQUOTA

Caps the namespace

- Totals for the whole namespace
- CPU, memory, storage
- Counts: Pods, Services, PVCs
- Over the cap: creation rejected

LIMITRANGE

Shapes each container

- Rules per container or Pod
- Default requests and limits
- Minimum and maximum sizes
- Fills gaps in Pod specs

THE TIEBREAKER

Quota caps the namespace. LimitRange shapes each container. They are often used together.

Quota and defaults for the payments team

```yaml
apiVersion: v1
kind: ResourceQuota
metadata: {name: team-quota, namespace: payments}
spec:
  hard: {requests.cpu: "8", requests.memory: 16Gi, pods: "40"}
---
apiVersion: v1
kind: LimitRange
metadata: {name: defaults, namespace: payments}
spec:
  limits:
  - type: Container
    defaultRequest: {cpu: 250m, memory: 256Mi}
    default: {cpu: 500m, memory: 512Mi}
```

5 The whole namespace: 8 CPUs of requests, 16 GiB, 40 Pods.

13-14 Requests and limits filled in for containers that set none.

The quota trap

- **With a CPU or memory quota,** every new Pod must declare requests or limits for it.
- **Pods without them are rejected,** and the Deployment shows no Pods at all.
- **A LimitRange with defaults** fixes this by filling the values in automatically.
- **Where to look:** the ReplicaSet's events show FailedCreate, with the quota reason.

> ON THE EXAM
>
> Deployment with zero Pods in a quota'd namespace: describe the ReplicaSet and read its events.

Keeping clusters healthy over time

- **Back up etcd** on a schedule, and practise restoring it.
- **Renew certificates** before they expire, or let regular upgrades do it.
- **Patch nodes by rotation:** drain, patch or replace, uncordon, one at a time.
- **Cluster API** manages whole clusters declaratively, as Kubernetes objects.

> ON THE EXAM
>
> Treat clusters like workloads: declared, versioned and replaceable.

> Quayside's control plane runs Kubernetes 1.37 on every API server. The team is reviewing the kubelet versions across its nodes.
>
> One of the node pools below is running an unsupported combination.

Which kubelet version is not supported with this control plane?

A kubelet 1.34

B kubelet 1.38

C kubelet 1.36

D kubelet 1.35

◆ PAUSE AND DECIDE

Which kubelet version is not supported with this control plane?

A kubelet 1.34

Three minors older: the oldest allowed.

B kubelet 1.38

Newer than the API server, which is never allowed.

C kubelet 1.36

One minor older: allowed.

D kubelet 1.35

Two minors older: allowed.

THE SIGNAL

Kubelets may lag by three minors, but may never lead.

> An engineer must patch the kernel on node-3 and reboot it. No new Pods should be placed there, and the Pods already on it must move elsewhere first.
>
> The booking API's disruption budget must be respected.

Which command does this?

A kubectl drain node-3 --ignore-daemonsets

B kubectl cordon node-3

C kubectl delete node node-3

D kubectl taint nodes node-3 maint=true:NoSchedule

◆ PAUSE AND DECIDE

Which command does this?

A kubectl drain node-3 --ignore-daemonsets

Cordons, then evicts gracefully through the budget-aware Eviction API.

B kubectl cordon node-3

Stops new Pods, but the existing ones stay put.

C kubectl delete node node-3

Removes the node object; Pods are not evicted through their budget.

D kubectl taint nodes node-3 maint=true:NoSchedule

Like cordon: no new Pods, but existing ones stay.

THE SIGNAL

'No new Pods and move the existing ones' is drain. Cordon does only half.

> A drain of node-5 hangs, repeating: cannot evict pod as it would violate the pod's disruption budget.
>
> The booking API runs 2 replicas, and its PodDisruptionBudget sets minAvailable: 2.

What is the cause, and the best fix?

A Drain is stuck; rerun it with --force

B DaemonSet Pods block it; add --ignore-daemonsets

C The node is cordoned twice; uncordon it once

D The budget allows no evictions; add a replica

◆ PAUSE AND DECIDE

What is the cause, and the best fix?

A. Drain is stuck; rerun it with --force
Force affects bare Pods; the budget still blocks eviction.

B. DaemonSet Pods block it; add --ignore-daemonsets
A real flag, but for a different error message.

C. The node is cordoned twice; uncordon it once
Cordoning is not counted. Uncordoning undoes the drain.

D. The budget allows no evictions; add a replica
With 3 replicas and minAvailable 2, one Pod can be evicted.

THE SIGNAL

minAvailable equal to replicas means zero allowed disruptions. Add headroom or relax the budget.

> The payments namespace has a ResourceQuota on requests.cpu and requests.memory. It has no LimitRange.
>
> A new Deployment is created, but no Pods ever appear. Its ReplicaSet shows FailedCreate: must specify requests.cpu.

What is the best fix?

A Raise the namespace's CPU quota

B Deploy it into the default namespace

C Add CPU and memory requests to the containers

D Delete the ResourceQuota from the namespace

◆ PAUSE AND DECIDE

What is the best fix?

A Raise the namespace's CPU quota

The quota is not full; the Pods simply lack requests.

B Deploy it into the default namespace

Dodges the team's controls rather than meeting them.

C Add CPU and memory requests to the containers

Meets the quota's requirement. A LimitRange with defaults also works.

D Delete the ResourceQuota from the namespace

Removes the protection that exists for good reason.

THE SIGNAL

'Must specify requests' under a quota: set requests, or add LimitRange defaults.

Module 11 in five lines

- kubeadm **init and join**; you must add a **CNI plugin**. Certificates last a year.
- Three minors maintained. **Kubelets never newer**, up to three minors older.
- Upgrade **control plane first**, one minor at a time, after an etcd backup.
- **Cordon** stops new Pods; **drain** also evicts, respecting PDBs.
- **ResourceQuota caps the namespace; LimitRange shapes each container.**

Quotas decide how much a team may use. Next: **scheduling**, which decides where each Pod runs.